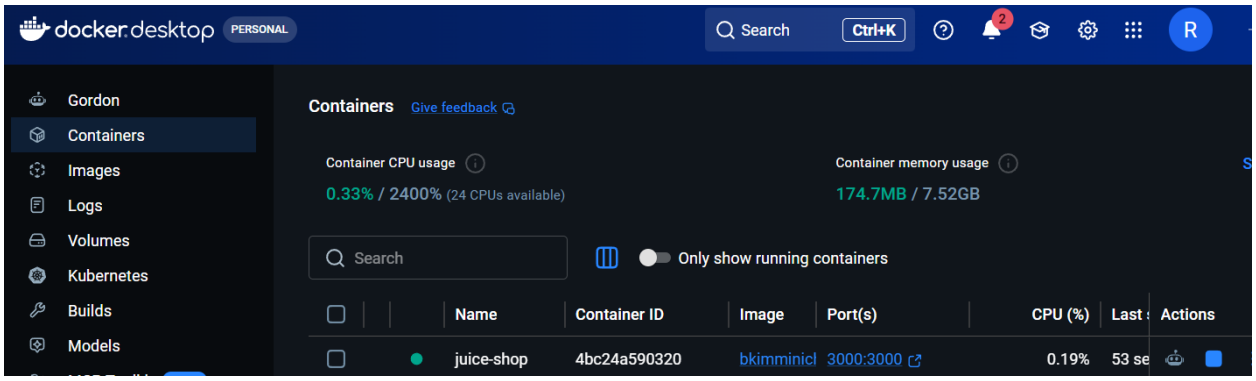
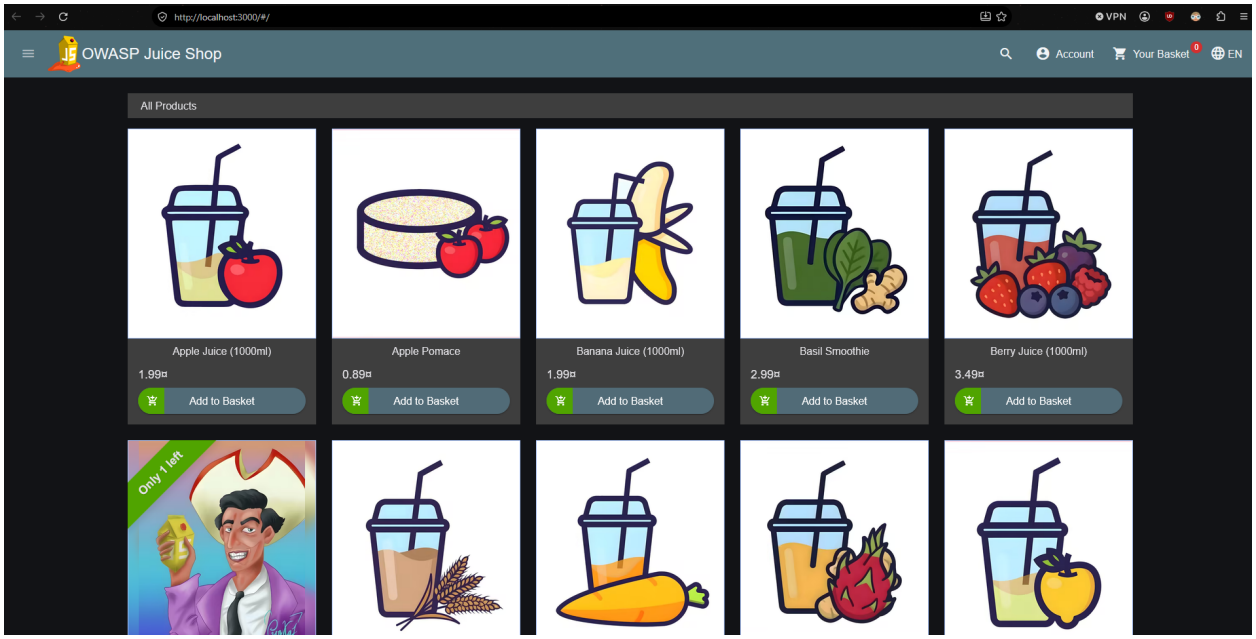
Setup

```
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Lab 5> docker run --rm -d --name juice-shop -p 3000:3000 bkimmi
nich/juice-shop
4bc24a590320a039c133df3eece794ff1041670216f2178ae45788c6c722e499
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Lab 5> docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS          PORTS
                    NAMES
4bc24a590320   bkimminich/juice-shop    "/nodejs/bin/node /j…"   26 seconds ago   Up 26 seconds   0.0.0.0:3000->3000/tcp,
 [::]:3000->3000/tcp   juice-shop
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Lab 5>
```





Part A

A1. Your first request

`requests.get(url, timeout=5)` sends a GET request. The `response` variable holds the result. `timeout=5` prevents Python from waiting indefinitely.

```python
import requests

base_url = "http://localhost:3000"
response = requests.get(base_url, timeout=5)

print("HTTP status:", response.status_code)
print("Response length:", len(response.content), "bytes")
```

✓ 0.1s

```
HTTP status: 200
Response length: 9984 bytes
```

A2. Reading response headers and content

`response.headers` acts like a dictionary of header names and values. `.get("Content-Type"` characters, rather than printing an entire web page.

```
print("Content type:", response.headers.get("Content-Type"))
print("First 100 characters:", response.text[:100])
```

0.0s

```
Content type: text/html; charset=UTF-8
First 100 characters: <!--
  ~ Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.
  ~ SPDX-Lice
```

A3. Sending a search parameter

The `params` argument builds the query string safely. The next request searches for the word `apple`. Notice that `requests` handles URL encoding for you.

```
search_url = base_url + "/rest/products/search"
r = requests.get(search_url, params={"q": "apple"}, timeout=5)

print("Requested URL:", r.url)
print("Status:", r.status_code)
print("Content type:", r.headers.get("Content-Type"))
print("First 150 characters:", r.text[:150])
```

[4] 0.0s Python

```
Requested URL: http://localhost:3000/rest/products/search?q=apple
Status: 200
Content type: application/json; charset=utf-8
First 150 characters: {"status":"success","data":[{"id":1,"name":"Apple Juice (1000ml)","description":"The all-time classic.","price":1.99,"deluxePrice":0.99,"image":"apple
```

Practice A:

A3. Sending a search parameter

The `params` argument builds the query string safely. The next request searches for the word `apple`. Notice that `requests` handles URL encoding for you.

```
search_url = base_url + "/rest/products/search"
r = requests.get(search_url, params={"q": "banana"}, timeout=5)

print("Requested URL:", r.url)
print("Status:", r.status_code)
print("Content type:", r.headers.get("Content-Type"))
print("First 150 characters:", r.text[:150])
```

0.0s Python

```
Requested URL: http://localhost:3000/rest/products/search?q=banana
Status: 200
Content type: application/json; charset=utf-8
First 150 characters: {"status":"success","data":[{"id":6,"name":"Banana Juice (1000ml)","description":"Monkeys love it the most.","price":1.99,"deluxePrice":1.99,"image":"
```

The query parameter q changed from apple to banana, so the server received a different search term. The status code may remain the same if both requests are processed successfully, although the actual result should be checked by running the code. The response length may differ because different search terms can return different products or amounts of content. A different response length alone does not prove a vulnerability; it may simply reflect normal application behavior

A4. GET versus POST

GET typically retrieves information; POST typically submits data. The HTTP method alone does not determine whether a request is secure. In this demonstration, POST is sent to the search path **only to observe how the server handles an unsupported method**. A 404 or 405 is an ordinary outcome.

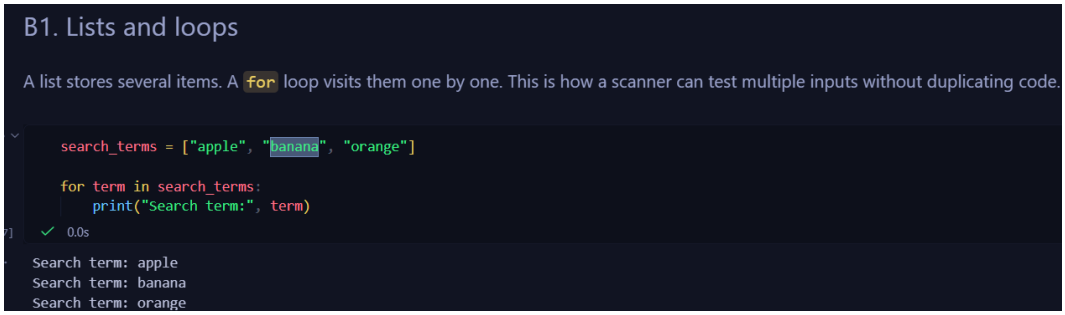
```
post_response = requests.post(
    search_url,
    data={"q": "apple"},
    timeout=5
)
print("POST status:", post_response.status_code)
print("POST response length:", len(post_response.content))
```

0.0s Python

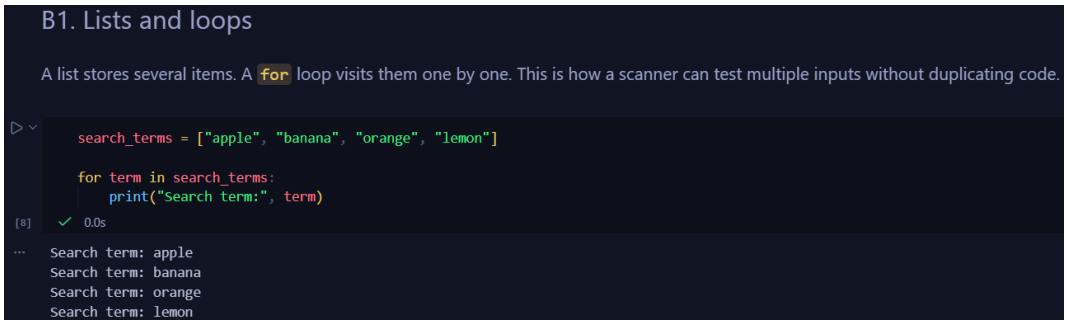
```
POST status: 500
POST response length: 2442
```

Practice B: The GET request attempts to retrieve search results, while the POST request submits data to the same search path using a different HTTP method. The server may process GET successfully but reject POST if that method is not supported for the endpoint. A 405 Method Not Allowed response can be normal behavior because the server is restricting the methods allowed for that resource. It does not automatically indicate a security vulnerability.

Part B



Practice C



The loop runs four times, once for each item in the list.



Practice D:

B2. Dictionaries: one record per request

A dictionary groups named values. For example, an HTTP observation can include `method`, `endpoint`, `status_code` and `response_length`. This is easier t
unnamed values.

```
observation = {
    "method": "GET",
    "endpoint": "/rest/products/search",
    "status_code": r.status_code,
    "response_length": len(r.content),
    "content_type": r.headers.get("Content-Type")
}

print(observation)
print("Status:", observation["status_code"])
```

0.0s

```
{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 504, 'content_type': 'application/json; charset=utf-8'}
Status: 200
```

B3. Combining a loop and a list of records

`append()` adds a new dictionary to a list. Notice that this example uses only two search t

```
records = []

for term in ["apple", "banana"]:
    result = requests.get(search_url, params={"q": term}, timeout=5)
    records.append({"input": term, "status_code": result.status_code})

print(records)
```

0.0s

```
[{'input': 'apple', 'status_code': 200}, {'input': 'banana', 'status_code': 200}]
```

Practice E:

B3. Combining a loop and a list of records

`append()` adds a new dictionary to a list. Notice that this example uses only two search terms and records just two fields; **it is not the com**

```
records = []

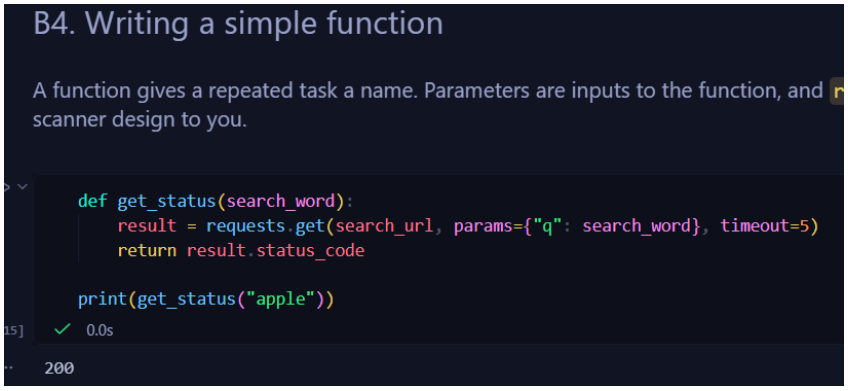
for term in ["apple", "banana"]:
    result = requests.get(search_url, params={"q": term}, timeout=5,)
    records.append({"input": term, "status_code": result.status_code, "response_length": len(result.content)})

print(records)
```

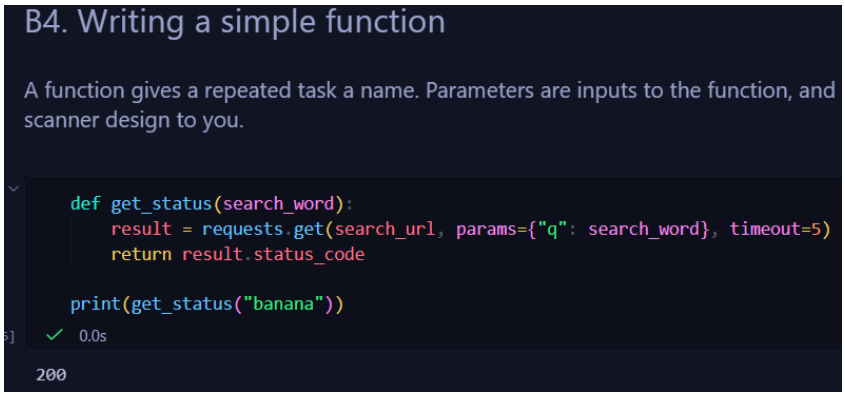
0.0s

```
[{'input': 'apple', 'status_code': 200, 'response_length': 1617}, {'input': 'banana', 'status_code': 200, 'response_length': 504}]
```

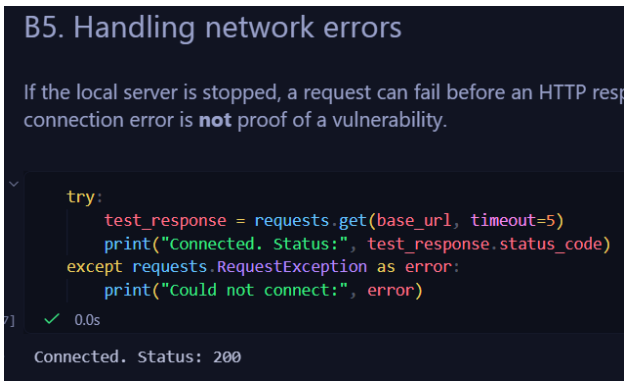
A list of dictionaries is useful because each dictionary represents one request and stores its details under descriptive field names. The records can be processed consistently and exported into a CSV file, making the results easier to review and compare.



Practice F



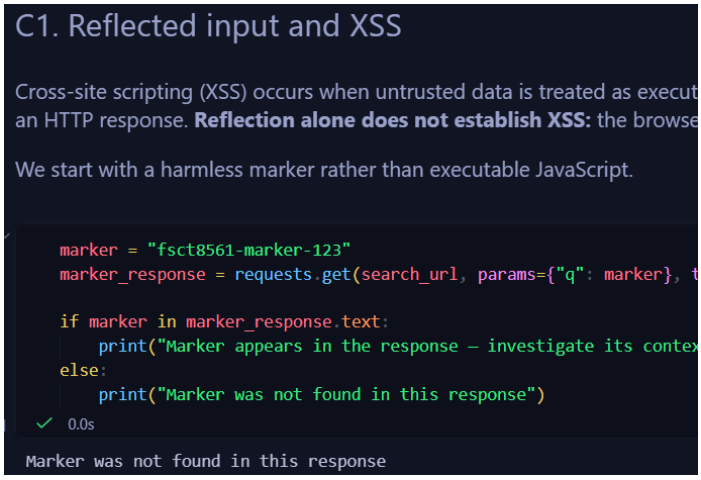
Calling get_status("banana") sends a GET request using banana as the search parameter and returns the HTTP status code received from the server. Inside the function, search_word is a parameter that receives the value passed to the function. This allows the same function to be reused with different search terms.



Practice G:

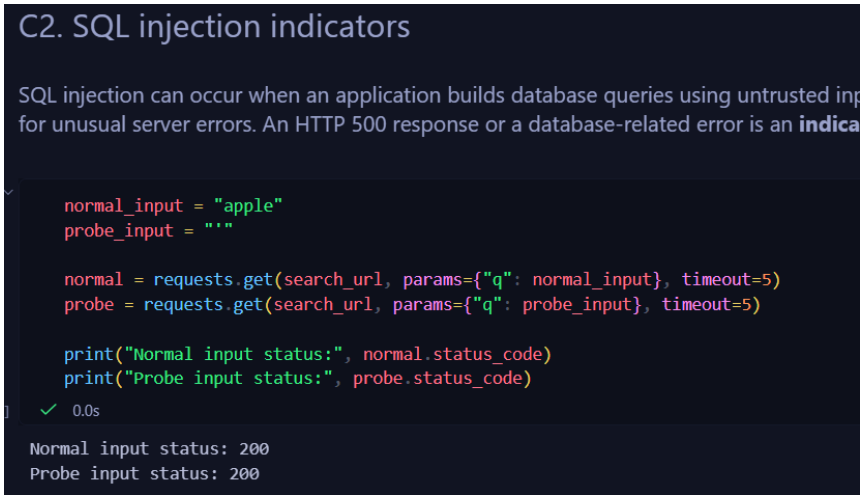
A timeout prevents a request from waiting indefinitely if the server is slow or unavailable. Handling requests.RequestException allows the program to report connection failures and other request-related errors without terminating unexpectedly. This makes the scanner more reliable and helps distinguish a connection problem from an HTTP response that might contain a security indicator.

Part C



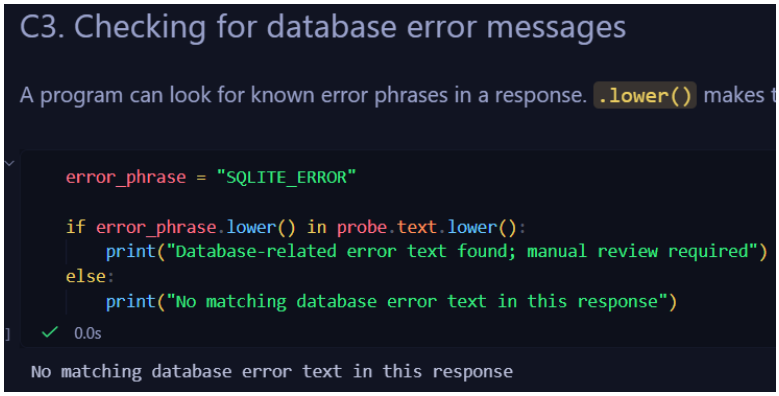
Practice H





Practice I:

The normal input, apple, provides a baseline for how the search endpoint behaves under ordinary conditions. The response to the single-quote probe can then be compared with that baseline for differences in status codes, response content, or error messages. This comparison helps identify unusual behavior associated with the probe. However, differences may result from normal input handling or validation, so they do not independently confirm SQL injection. Luckily for us, both returned with a 200 status OK.



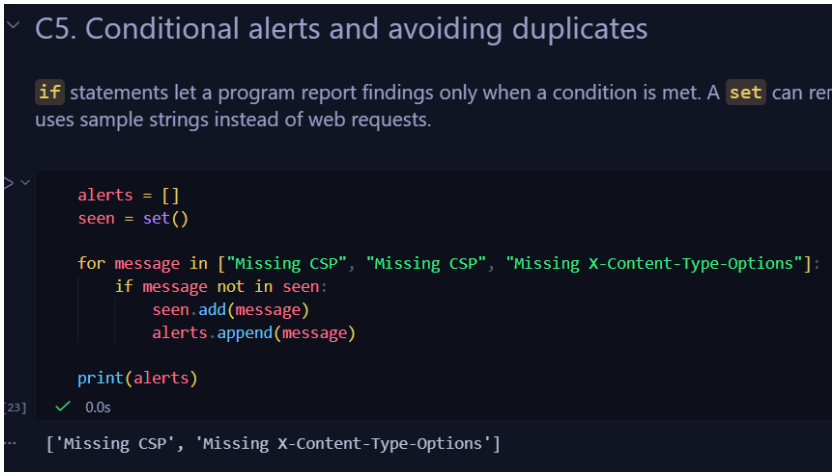
Practice J: Two more phrases a detector could check are syntax error and database error. Searching for these phrases can help identify responses that may contain database-related error information. However, an error phrase alone does not prove SQL injection because the message may be generic, unrelated to the tested input, or produced by another application component. The finding should be recorded as an indicator requiring manual verification.



Practice K: The scan found that the Content-Security-Policy (CSP) header is not present, so its relevance should be investigated. CSP helps restrict which resources a webpage can load and provides an additional layer of protection against cross-site scripting (XSS). Its absence may reduce browser-side protection, but it does not automatically mean the website is vulnerable.

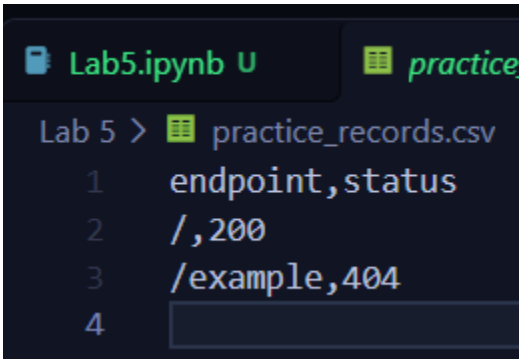
The X-Content-Type-Options header is present, which is a positive security indicator. When configured as nosniff, it tells browsers to respect the declared content type rather than guessing the type of a resource, helping reduce certain content-sniffing risks.

The missing CSP header should be reviewed, while the presence of X-Content-Type-Options is a positive finding. These results alone do not establish whether the application is secure or vulnerable.



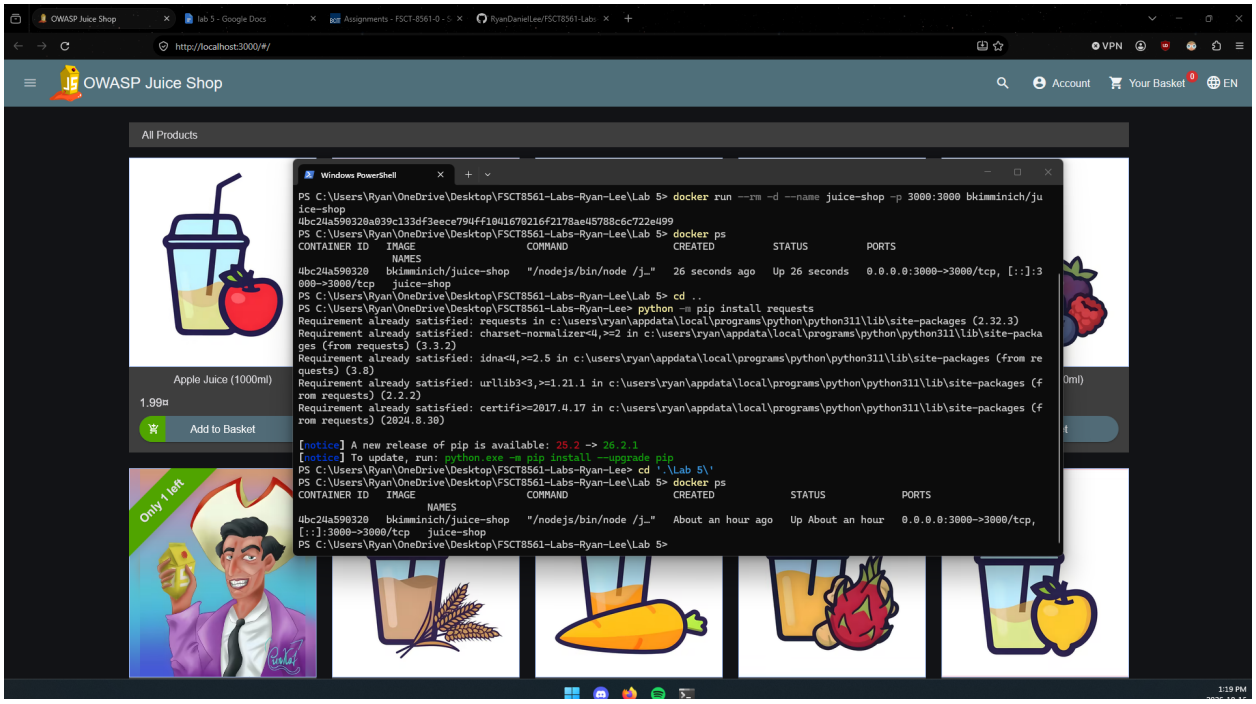
Practice L: The list contains only two alerts because the message Missing CSP appears twice, and the seen set tracks messages that have already been processed. When the duplicate appears, it is not added to the alert list again. The other message, Missing X-Content-Type-Options, is unique and is added. In the final detector, duplicate findings can be identified using a combination of the finding category, endpoint, and observation so that repeated reports of the same issue are avoided.





Practice M: The CSV file contains two column headings: endpoint and status. It has two data rows: / with status 200, and /example with status 404. CSV is useful for the final report because it organizes results into rows and columns, making the observations easy to inspect, compare, and include as evidence. It can also be opened in spreadsheet applications such as Microsoft Excel or Google Sheets.

Part 1, 2, 3 and 4



Http Scanner

```
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Lab 5> python .\http_scanner.py
[GET] Endpoint: / | Input: None | Status: 200 | Length: 9984 bytes
[GET] Endpoint: /rest/products/search | Input: {'q': 'apple'} | Status: 200 | Length: 1617 bytes
[GET] Endpoint: /rest/products/search | Input: {'q': 'banana'} | Status: 200 | Length: 504 bytes
[POST] Endpoint: /rest/products/search | Input: {'q': 'apple'} | Status: 500 | Length: 2442 bytes
Saved http_scan_results.csv successfully.

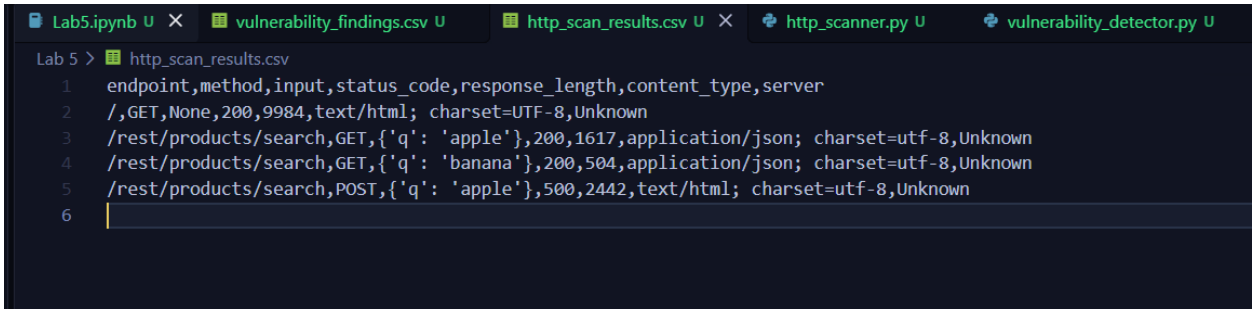
--- VULNERABILITY DETECTOR SUMMARY ---
Category:      SQL Injection Indicator
Endpoint:      /rest/products/search
Observation:   Probe ''')' returned status 500 (Normal status 200) | DB Error text found: True
Review Status: Indicator found (HTTP 500 or DB error text); manual review required, not proof of exploitation
----------------------------------------
Category:      Configuration Review
Endpoint:      /
Observation:   Header 'Content-Security-Policy' is missing or empty
Review Status: Review configuration; this alone does not prove exploitation
----------------------------------------
Saved vulnerability_findings.csv successfully.
```

Vulnerability Scanner

```
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Lab 5> python .\vulnerability_detector.py

--- VULNERABILITY DETECTOR SUMMARY ---
Category:      SQL Injection Indicator
Endpoint:      /rest/products/search
Observation:   Probe ''')' returned status 500 (Normal status 200) | DB Error text found: True
Review Status: Indicator found (HTTP 500 or DB error text); manual review required, not proof of exploitation
----------------------------------------
Category:      Configuration Review
Endpoint:      /
Observation:   Header 'Content-Security-Policy' is missing or empty
Review Status: Review configuration; this alone does not prove exploitation
----------------------------------------
Saved vulnerability_findings.csv successfully.
PS C:\Users\Ryan\OneDrive\Desktop\FSCT8561-Labs-Ryan-Lee\Lab 5>
```

CSV Results





The HTTP scanner tested the local OWASP Juice Shop application at http://localhost:3000. The homepage returned HTTP 200, and both product searches for “apple” and “banana” also returned HTTP 200. However, the POST request to /rest/products/search returned HTTP 500, indicating that the server encountered an internal error when processing the request. This result should be investigated, but it does not mean there is a security vulnerability.
The vulnerability detector checked the homepage for the Content-Security-Policy and X-Content-Type-Options headers. The detector reported that Content-Security-Policy was missing or empty. CSP is a security mechanism that can restrict the sources from which browsers load resources and help reduce certain cross-site scripting (XSS) risks. Its absence is worth reviewing, although it does not mean that the application is vulnerable. The detector did not report X-Content-Type-Options as missing, suggesting that the header was present and non-empty. Its actual value should be checked to confirm whether it is configured appropriately, such as using nosniff. The scanner recorded the Server header as Unknown for all four requests. This means the header was not available in the responses, it does not really mean the server is misconfigured. When present, a Server header may disclose software or server implementation information that could help an attacker identify technologies to investigate. Finally, these tests used local HTTP rather than HTTPS. Therefore, they do not meaningfully assess TLS certificate validity, encryption, or HTTPS configuration. A missing header or an HTTP 500 response should be treated as an observation requiring further investigation, not proof of an exploitable vulnerability.

Part 5

The automated HTTP scanner successfully tested the local OWASP Juice Shop homepage and product search endpoint. The homepage and both ordinary searches, “apple” and “banana,” returned HTTP 200. The response lengths differed because the returned content was different.

The POST request to the product search endpoint returned HTTP 500, showing that the server encountered an error when processing that request. This behavior should be investigated further to determine whether the endpoint rejects the method or has an application-handling problem.

The vulnerability detector identified two findings. First, the SQL syntax probe ') returned HTTP 500, compared with HTTP 200 for the normal search, and the response contained text matching a database-error phrase. This is a potential SQL injection indicator, but it does not prove that SQL injection is possible. The response and application logs would need further investigation. Second, the detector reported that the homepage lacked a Content-Security-Policy header. CSP can help restrict browser resource loading and reduce some XSS risks, so reviewing the application's security configuration would be worthwhile. The scanner also recorded the Server header as unavailable. The actual value of X-Content-Type-Options should be checked to verify its configuration.

I would investigate the server errors first, review the missing CSP header, and manually verify each finding before deciding its severity. Automated tools can produce false positives, so their results should guide further testing rather than be treated as confirmed vulnerabilities.

This lab extends passive packet analysis from Lab 4 by actively sending application-layer HTTP requests and examining responses, status codes, headers, and error messages. Packet capture can help observe network traffic, while an authorized Python client can systematically test application behavior. Because these tests used HTTP on localhost, they do not evaluate HTTPS encryption or TLS certificate security. Overall, the lab demonstrated how automation can collect evidence efficiently while still requiring careful interpretation and manual verification.